

INDIRA GANDHI NATIONAL OPEN UNIVERSITY

School of Computer and Information Sciences

Maidan Garhi, New Delhi – 110 068

Project Report

Course Code: **BCSP-064**

BCA – 6th Semester

Hotel Room Booking Website

with Assistant Chatbot

Submitted in partial fulfilment of the requirement for the award of the degree
of

Bachelor of Computer Applications (BCA)

Submitted By

Name: **Nikhil P N**

Enrolment No: **2350526613**

Programme: BCA

Study Centre Code: _____

Regional Centre: **IGNOU Kolkata Regional Centre**

Address: _____

Under the Guidance of

Guide Name: _____

Designation: _____

Qualification: _____

Address: _____

Academic Year: **2025 – 2026**

Certificate of Originality

This is to certify that the project report entitled "**Hotel Room Booking Website with Assistant Chatbot**" submitted to **Indira Gandhi National Open University (IGNOU)** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (BCA)** is an original piece of work carried out by **Mr. Nikhil P N**, bearing Enrolment Number **2350526613**, under the guidance of **Mr./Ms. _____**.

The matter embodied in this project report is a genuine work done by the student and has not been submitted in part or in full, whether to this University or to any other University or Institute, for the fulfilment of the requirement of any course of study.

It is further certified that the student has fulfilled all the conditions of eligibility for submitting the project report, the report has been examined, and I recommend the same for the award of the degree of BCA.

Signature of the Student

Name: Nikhil P N

Enrolment No: 2350526613

Date: _____

Signature of the Guide

Name: _____

Designation: _____

Address: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to **Indira Gandhi National Open University (IGNOU)** for providing me with the opportunity to undertake this project as part of the BCA programme. The knowledge and skills gained during the course of study have been instrumental in the successful completion of this project.

I am deeply indebted to my project guide, **Mr./Ms. _____**, whose valuable guidance, constant encouragement, and constructive suggestions throughout the duration of this project enabled me to complete it successfully. Their expertise in the field of computer science and willingness to spare their valuable time has been immensely helpful.

I extend my heartfelt thanks to the **IGNOU Kolkata Regional Centre** and my Study Centre for their administrative support and for ensuring the smooth progression of my academic journey throughout the BCA programme.

I also wish to acknowledge my gratitude to all the faculty members of the School of Computer and Information Sciences, IGNOU, whose teachings in subjects like Database Management Systems, Internet Technology, Software Engineering, Object-Oriented Technology, and Web Programming have provided me with the theoretical foundation and practical skills necessary for this project.

I am also thankful to the open-source community, the developers of the Django

framework, the Python programming language, and the various libraries and tools (Razorpay, xhtml2pdf, Bootstrap) used in this project. Their freely available documentation, tutorials, and forums have been invaluable resources.

Last but not least, I express my deepest gratitude to my family and friends for their constant moral support and encouragement throughout the period of this project work. Their faith in me kept me motivated even during challenging phases of development and testing.

Nikhil P N

Enrolment No: 2350526613

BCA – 6th Semester

IGNOU Kolkata Regional Centre

Table of Contents

1. Certificate of Originality
2. Acknowledgement
3. Abstract
4. Introduction
 - Background of the Problem
 - Motivation for the Project
 - Scope of the Project
5. Objectives of the Project
6. Project Category
7. System Study
 - Study of Existing System
 - Limitations of the Existing System
 - Proposed System
 - Advantages of the Proposed System
8. Feasibility Study
 - Technical Feasibility
 - Operational Feasibility
 - Economic Feasibility
9. Tools / Platform, Hardware and Software Requirements
10. Analysis Document
 - Software Requirement Specification (SRS)
 - Data Flow Diagrams (DFD Level-0 & Level-1)

- Entity-Relationship Diagram
- Data Dictionary

11.Design Document

- Modularization Details (8 Modules)
- Database Design and Normalization
- Procedural Design / Process Logic
- User Interface Design

12.Source Code (Key Modules)

- Django Models
- Booking with Overlap Detection
- Advanced Search API
- Chatbot Engine
- Payment Integration
- OTP-Based Password Reset

13.Testing

- Testing Strategy
- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing
- Debugging and Bug Fixing

14.Validations Performed

15.Input and Output Screens

16.Report Generation

17.Implementation of Security

18. Deployment and Installation Guide

19. Limitations of the Project

20. Future Scope and Enhancement

21. Glossary of Terms

22. Bibliography

1. Abstract

The **Hotel Room Booking Website with Assistant Chatbot** is a comprehensive, full-stack web application developed using **Python**, the **Django 6.0** web framework, and a **PostgreSQL** relational database. The system has been designed and developed to automate end-to-end hotel operations including room-type management, room inventory control, customer registration, online room booking with date-based availability checking, payment processing through the Razorpay payment gateway, staff administration, customer contact handling, email notifications, PDF report generation, and an intelligent assistant chatbot that can answer guest queries about rooms, bookings, staff, and hotel policies.

The project follows the **Software Development Life Cycle (SDLC)** and encompasses all the major phases: Requirements Analysis, System Design, Coding, Testing, and Documentation. It consists of eight functional modules spanning both the administrator backend and the customer-facing website, and exposes RESTful API endpoints for the chatbot and advanced search features. The application is fully responsive and works seamlessly across desktop computers, tablets, and mobile devices.

The administrator can manage room types, rooms, staff profiles, customer enquiries, bookings, payments, chatbot responses, and notifications from a secure backend interface. Customers can register, sign in, browse rooms by category, search with advanced filters (room type, price range, dates), book

available rooms with automatic overlap detection, make secure online payments via Razorpay, interact with the chatbot for instant assistance, submit contact enquiries, and reset their passwords using email-based OTP verification.

The system stores data in a normalized relational database with ten interconnected tables and enforces referential integrity through primary-key and foreign-key constraints. Efficient querying, input validation, CSRF protection, and session-based access control ensure a secure and responsive user experience. The project demonstrates the practical application of RDBMS concepts, internet technology, and AI-assisted user interaction in the hospitality domain.

Keywords: Hotel Room Booking, Django, Python, PostgreSQL, Chatbot, Razorpay, AJAX, Bootstrap, PDF Generation, Email OTP, REST API, SDLC, Software Engineering.

2. Introduction

2.1 Background of the Problem

The hospitality industry faces challenges in managing room inventory, bookings, payments, and guest communication. Many small hotels still use manual paper registers or spreadsheets, leading to duplicate bookings, inaccurate availability records, delayed communication, and difficulty generating reports. Customers increasingly expect online booking, real-time availability, and instant 24/7 support through chatbots. This project addresses these challenges with a unified web-based platform.

2.2 Motivation for the Project

The motivation stems from the BCSP-064 academic requirement, the real-world need for affordable hotel booking solutions, the opportunity to develop full-stack skills (Django, Python, JS, PostgreSQL), and the growing importance of chatbot-based customer service in the hospitality domain.

2.3 Scope of the Project

The scope covers: (1) a responsive customer-facing website with room browsing, advanced search, booking, Razorpay payment, chatbot, and OTP password reset; (2) a secure admin backend for managing room types, rooms, staff, bookings, payments, contacts, chatbot responses, and notifications; (3) a

normalized PostgreSQL database with 10 tables; (4) RESTful API endpoints for chatbot, search, and notifications; and (5) PDF and HTML reporting. Multi-hotel management, dynamic pricing, loyalty programs, and OTA integration are identified as future enhancements.

3. Objectives of the Project

1. Design a web-based hotel room booking system using Django and Python.
2. Provide responsive customer interface for browsing and searching rooms.
3. Implement date-range overlap detection to prevent double booking.
4. Integrate Razorpay payment gateway for secure online transactions.
5. Build an AI-assistant chatbot using rule-based keyword classification.
6. Implement AJAX-powered advanced room search.
7. Provide email notifications for booking confirmation and OTP password recovery.
8. Generate downloadable PDF reports of booking details.
9. Ensure secure access via authentication, CSRF, and input validation.
10. Follow SDLC practices throughout development.

4. Project Category

This project spans three IGNOU categories:

Category	Description
RDBMS	PostgreSQL with 10 normalized tables, foreign keys, Django ORM.
Internet Technology	Full-stack Django app with HTML5, CSS3, Bootstrap 5, JS/AJAX, REST APIs.
AI Application	Rule-based chatbot with keyword classification, DB queries, user ratings.

5. System Study

5.1 Study of the Existing System

Most small and medium hotels use paper registers, phone-based reservations, or spreadsheets. These methods lack real-time availability, require manual data entry, and make reporting difficult.

5.2 Limitations

Double booking risk, data redundancy, slow retrieval, no online access, no automated communication, delayed payment processing, no 24/7 support.

5.3 Proposed System

Web-based Django application with centralized PostgreSQL database, admin CRUD, customer booking/search, Razorpay payment, chatbot, email notifications, and PDF reports.

5.4 Advantages

Eliminates double bookings, centralized data, AJAX search, automated reporting, session security, 24/7 online access, chatbot support, secure payments.

6. Feasibility Study

6.1 Technical Feasibility

Conclusion: Technically feasible. Python, Django, PostgreSQL, Bootstrap, Razorpay are all mature, open-source, and within the developer's skill set.

6.2 Operational Feasibility

Conclusion: Operationally feasible. Simple admin forms, familiar customer interface, minimal training needed, chatbot reduces staff workload.

6.3 Economic Feasibility

Conclusion: Economically feasible. All tools are free/open-source. Only cost is web hosting (₹200-500/month). Benefits far outweigh negligible development cost.

7. Tools / Platform, Hardware and Software Requirements

7.1 Software Requirements

Component	Specification
Operating System	Windows 10/11, Linux (Ubuntu 20.04+), or macOS
Programming Language	Python 3.10+ (3.x series)
Web Framework	Django 6.0
Database	PostgreSQL 16 (open-source, advanced relational database)
Frontend – Markup & Styling	HTML5, CSS3, Bootstrap 5.3
Frontend – Scripting	JavaScript (ES6), AJAX (Fetch API)
Payment Gateway	Razorpay Python SDK (razorpay package)
PDF Generation	xhtml2pdf 0.2+ (Python library)
Email Backend	Django SMTP Email Backend (Gmail SMTP)
Environment	python-decouple (for managing secrets via .env)

Configuration	file)
IDE / Code Editor	Visual Studio Code (VS Code)
Web Browser (Testing)	Google Chrome 120+, Microsoft Edge 120+, Mozilla Firefox 120+
Version Control	Git 2.x
Virtual Environment	Python venv module

7.2 Python Package Dependencies

The following Python packages are installed in the virtual environment (as specified in `requirements.txt`):

Package	Purpose
<code>django</code>	Web framework
<code>razorpay</code>	Razorpay payment gateway SDK
<code>xhtml2pdf</code>	HTML to PDF conversion
<code>reportlab</code>	PDF rendering engine (dependency of <code>xhtml2pdf</code>)
<code>python-decouple</code>	Environment variable management
<code>Pillow</code>	Image processing for <code>ImageField</code>
<code>pdfplumber</code>	PDF text extraction utility

7.3 Hardware Requirements

Component	Minimum Specification	Recommended
Processor	Intel Core i3 / Dual-core equivalent	Intel Core i5 or above
RAM	4 GB	8 GB or above
Hard Disk Space	20 GB free	50 GB SSD
Display Resolution	1366 × 768	1920 × 1080 (Full HD)
Internet Connection	Required (for payment gateway, email, and deployment)	Broadband 10 Mbps+
Input Devices	Keyboard and Mouse	Standard peripherals

8. Analysis Document

8.1 Software Requirement Specification (SRS)

8.1.1 Purpose

The purpose of this Software Requirement Specification is to describe the functional and non-functional requirements of the Hotel Room Booking Website with Assistant Chatbot. This document serves as a reference for the development team and the project guide, and defines the expected behaviour of the system.

8.1.2 Scope

The system is a web-based application for hotel room booking and management. It serves two categories of users: **Administrators** (hotel staff) and **Customers** (guests). The application will be accessible via a web browser and will communicate with a PostgreSQL database on the server.

8.1.3 Functional Requirements – Administrator

Req. ID	Requirement	Description
FR-A01	Admin Login	The admin shall be able to log in using Django's built-in authentication system (username and password).

FR-A02 Admin Logout	The admin shall be able to securely log out, destroying the session.
FR-A03 Add Room Type	The admin shall be able to add a new room type with name, description, and image.
FR-A04 View Room Types	The admin shall be able to view all room types in a tabular format with edit and delete actions.
FR-A05 Edit Room Type	The admin shall be able to modify the name, description, and image of an existing room type.
FR-A06 Delete Room Type	The admin shall be able to delete a room type (subject to referential integrity).
FR-A07 Add Room	The admin shall be able to add a new room linked to a room type, with name, price, description, and image.
FR-A08 View Rooms	The admin shall be able to view all rooms with their type, price, and actions.
FR-A09 Edit Room	The admin shall be able to modify room details.
FR-A10 Delete Room	The admin shall be able to delete a room (subject to referential integrity).
FR-A11 Add Staff	The admin shall be able to add staff with name,

designation, social links, and photo.

FR-A12	View Staff	The admin shall view all staff with details and actions.
FR-A13	Edit Staff	The admin shall be able to modify staff details.
FR-A14	Delete Staff	The admin shall be able to delete a staff record.
FR-A15	View Contact Messages	The admin shall view all customer contact messages.
FR-A16	Delete Contact Messages	The admin shall be able to delete processed contact messages.
FR-A17	View Bookings	The admin shall view all booking records with customer, room, dates, and price.
FR-A18	View Payment Totals	The admin shall view all payment records with customer, mobile, and total amount.
FR-A19	Manage Chatbot	The admin shall manage chatbot responses, conversations, and notifications via Django Admin.

8.1.4 Functional Requirements – Customer

Req. ID	Requirement	Description
FR-C01	User Registration	A new customer shall be able to register with

		username, email, password, and confirm password.
FR-C02	User Sign-In	A registered customer shall be able to sign in with username and password.
FR-C03	User Sign-Out	A signed-in customer shall be able to sign out, destroying the session.
FR-C04	Browse Room Categories	The customer shall see room type cards on the home page and navigate to rooms of a specific type.
FR-C05	View Room Details	The customer shall see room name, image, price, description, and a "Book" button for each room.
FR-C06	Advanced Search	The customer shall search rooms by name, type, min price, max price, check-in date, and check-out date using an AJAX-powered search panel.
FR-C07	Book a Room	The customer shall fill in booking details (name, email, dates, adults, children, special request) and submit a booking.
FR-C08	Overlap Detection	The system shall check for overlapping bookings and reject the booking if the room is already booked for the requested dates.
FR-C09	View Booking Cart	The customer shall see booked rooms with subtotal, tax, and total amount.

FR-C10	Delete Booking	The customer shall be able to remove a booking from the cart before payment.
FR-C11	Make Payment	The customer shall pay the total amount via Razorpay (credit/debit card, UPI, net banking).
FR-C12	Submit Contact Enquiry	The customer shall submit a contact form with name, email, phone, subject, and message.
FR-C13	Chatbot Interaction	The customer shall send text messages to the chatbot and receive contextual responses.
FR-C14	Rate Chatbot Response	The customer shall rate chatbot responses on a 1–5 scale.
FR-C15	Password Reset via OTP	The customer shall reset their password using an OTP sent to their registered email.
FR-C16	Receive Email Confirmation	The customer shall receive a booking confirmation email after successful booking.

8.1.5 Non-Functional Requirements

Category	Requirement
Performance	Web pages shall load within 2–3 seconds on a broadband connection. AJAX search results shall appear without a full page reload.

Security	CSRF protection on all forms, ORM-based parameterised queries to prevent SQL injection, template auto-escaping to prevent XSS, session-based authentication, password masking on input fields.
Usability	Responsive design that works on desktop (1920px), tablet (768px), and mobile (375px) screen widths. Consistent navigation and layout across all pages.
Reliability	Input validation on all forms (required fields, numeric checks, date format checks). Graceful error handling with user-friendly messages. Date-overlap detection prevents booking conflicts.
Maintainability	Modular Django app structure (Backend app, webapp). Separation of concerns (models, views, templates, URLs). Code documentation with comments.
Scalability	Database uses PostgreSQL for production-grade scalability, concurrent write support, and cloud deployment readiness. New modules can be added as new Django apps. API endpoints support future mobile app development.
Portability	The application runs on Windows, Linux, and macOS. It requires only Python, Django, and a web browser.

8.2 Data Flow Diagrams (DFD)

8.2.1 DFD Level-0 (Context Diagram)

The context diagram shows the system with two external entities: **Customer** and **Administrator**.

Entity	Input to System	Output from System
Customer	Registration, Login, Search,	Room details, Booking
	Booking, Payment, Contact,	confirmation, Search results,
	Chat messages	Chat responses, PDF
Administrator	Room/Staff/Type CRUD, View bookings, View payments	Reports, Records, Notifications, Chatbot logs

8.2.2 DFD Level-1

The Level-1 DFD decomposes the single process of the context diagram into eight sub-processes corresponding to the eight functional modules of the system. It also shows the data stores (database tables) that each process reads from and writes to.

Process	Input Data Flow	Output Data Flow	Data Store(s)
1.0 User	Username,	Session token,	D1: USER_MASTER
Registration &	Email,	Access	(Registerdb)
Login	Password,	granted/denied,	

	Login credentials	Success/Error message	
2.0 Room Type Management	Room type name, image, description	Updated room type catalogue, Success message	D2: ROOMTYPE_MASTER (roomtypedb)
3.0 Room Management	Room name, type FK, image, price, description	Updated room inventory, Success message	D3: ROOM_MASTER (roomnamedb), D2
4.0 Staff Management	Staff name, designation, image, social links	Staff list for website & admin	D4: STAFF_MASTER (staffdb)
5.0 Room Search & Booking	Search filters, dates, guest details	Search results (JSON), Booking confirmation/rejection, Email	D3, D5: BOOKING_MASTER (bookingdb), D1
6.0 Payment Processing	Booking total, customer mobile, payment	Razorpay order, Payment receipt, Success message	D6: PAYMENT_TOTAL (Totaldb), D5

	method		
7.0 Contact Management	Name, email, phone, subject, message	Stored enquiry, Success message	D7: CUSTOMER_CONTACT (customercontactdb)
8.0 Assistant Chatbot	User text message, Username	Classified response, Room cards, Suggestions	D8: ChatbotConversation, D9: ChatbotResponse, D3, D4, D5

Table 8.1 – DFD Level-1 Process Summary

8.3 Entity-Relationship (ER) Diagram

The ER diagram identifies the entities, their attributes, primary keys, and the relationships (with cardinality) between entities in the system.

8.3.1 Entities and Their Key Attributes

Entity	Primary Key	Key Attributes
roomtypedb	id (Auto PK)	ROOMTYPE, ROOMTYPEIMAGE, DESCRIPTION
roomnamedb	id (Auto PK)	ROOMTYPE (FK), ROOMNAME, ROOMIMAGE, ROOMPRICE, ROOMDESCRIPTION
Registerdb	id (Auto	USERNAME, PASSWORD,

	PK)	CONFIRMPASSWORD, EMAIL
		CUSTOMER (FK), CUSTOMERNAME,
bookingdb	id (Auto PK)	CONTACTEMAIL, CHECKIN, CHECKOUT, TOTALADULTS, TOTALCHILDS, SELECTROOM (FK), SPECIALREQUEST, TOTALPRICE
Totaldb	id (Auto PK)	BOOKING (FK), CUSTOMERNAME, MOBILE, TOTALPRICE
staffdb	id (Auto PK)	STAFFNAME, STAFFDESIGNATION, STAFFFACEBOOK, STAFFINSTA, STAFFIMAGE
customercontactdb	id (Auto PK)	CONTACTNAME, CONTACTNUMBER, CONTACTEMAIL, CONTACTSUBJECT, CONTACTMESSAGE
ChatbotConversation	id (Auto PK)	username, user_message, bot_response, query_type, rating, created_at
ChatbotResponse	id (Auto PK)	Predefined response templates
Notification	id (Auto PK)	username, notification_type, title, message, is_read, related_object_id, created_at

8.3.2 Relationships and Cardinality

Relationship	Cardinality	Description
--------------	-------------	-------------

roomtypedb ↔

roomnamedb

1 : N (One-to-Many)

One room type can have many rooms. Each room belongs to exactly one room type.

roomnamedb ↔

bookingdb

1 : N (One-to-Many)

One room can have many bookings (over time). Each booking is for exactly one room.

Registerdb ↔

bookingdb

1 : N (One-to-Many)

One registered user can make many bookings. Each booking is optionally linked to one user.

bookingdb ↔ Totaldb

1 : N (One-to-Many)

One booking can have payment records. Each payment is linked to one booking.

staffdb

Independent

No direct relationship with other entities.

customercontactdb

Independent

No direct relationship with other entities.

ChatbotConversation

Independent

Stores username as text field; no FK to Registerdb.

8.4 Data Dictionary

The database has 10 tables: roomtypedb, roomnamedb, Registerdb, bookingdb, Totaldb, staffdb, customercontactdb, ChatbotConversation, ChatbotResponse, and Notification.

Table	Key Fields	Relationships
roomtypedb	ROOMTYPE, ROOMTYPEIMAGE, DESCRIPTION	PK → roomnamedb.ROOMTYPE
roomnamedb	ROOMNAME, ROOMIMAGE, ROOMPRICE, ROOMDESCRIPTION	FK → roomtypedb; PK → bookingdb.SELECTROOM
Registerdb	USERNAME, PASSWORD, EMAIL	PK → bookingdb.CUSTOMER
bookingdb	CHECKIN, CHECKOUT, TOTALADULTS, TOTALCHILDS, TOTALPRICE	FK → Registerdb, roomnamedb
Totaldb	MOBILE, TOTALPRICE	FK → bookingdb
staffdb	STAFFNAME, STAFFDESIGNATION,	Independent

	STAFFIMAGE	
	CONTACTNAME,	
customercontactdb	CONTACTEMAIL,	Independent
	CONTACTSUBJECT,	
	CONTACTMESSAGE	
	user_message,	
ChatbotConversation	bot_response,	Independent
	query_type, rating	
	keywords,	
ChatbotResponse	response_text, category,	Independent
	is_active	
	notification_type, title,	
Notification	message, is_read	Independent

9. Design Document

9.1 Modularization Details – 8 Modules

The system is divided into eight functional modules. Each module handles a specific set of related features. The modules are described below with their purpose, functions, input/output, and pseudocode.

Module 1 – Secure Login & User Management

Purpose: This module allows only authorized users to access customer features or the admin backend. It verifies the username and password against the registered users table and stores the username in the session so that unauthorized persons cannot view, modify, or delete data.

Functions Covered:

- User registration (sign-up) with username, email, password, and confirm password
- User sign-in with session creation
- User sign-out with session destruction
- Admin login using Django's built-in `authenticate()` and `login()` functions
- Admin logout using Django's `logout()` function
- Password reset using email OTP verification
- OTP generation, email dispatch, OTP verification, and password update

Module 2 – Room Type & Room Management

Purpose: This module enables the admin to insert, update, and delete room types (e.g., Deluxe, Suite, Standard) and individual rooms including images, descriptions, and prices, so that the online catalogue always matches the actual inventory of the hotel. Rooms are linked to their room type via a foreign key relationship.

Functions Covered:

- Add room type with name, description, and image upload
- Display all room types in a table with edit/delete buttons
- Edit room type details and image
- Delete room type (cascade deletes associated rooms)
- Add room with name, type (dropdown FK), price, description, and image
- Display all rooms with type, price, and actions
- Edit room details
- Delete room

Module 3 – Staff Management

Purpose: When new staff members join or existing staff details change, this module is used. It allows the admin to maintain staff name, designation, social-media links (Facebook, Instagram), and photos. These details are displayed on the About and Our Team pages so that customers can know about the team behind the hotel services.

Functions Covered: Add staff, display all staff, edit staff, delete staff.

Module 4 – Customer Contact Management

Purpose: When a visitor sends a message through the contact form, this module stores the details—customer name, email, mobile number, subject, and message—so that the admin can read, respond to, and delete messages from a single backend page, improving hotel–guest communication.

Module 5 – Room Search & Booking Management

Purpose: This is one of the main functional modules. It allows a logged-in customer to search rooms by room type, room name, date range, and price and then place a booking request. The module checks availability by validating date-range overlaps for the selected room, prevents double booking, stores booking details such as check-in, check-out, adults, children, special requests, and calculates the total price. An advanced search interface using AJAX provides real-time filtering without full page reload.

Functions Covered:

- Browse rooms by category (filter by room type)
- Advanced search with AJAX (room name, type, min price, max price, check-in, check-out)
- Book a room with overlap detection
- View booking cart with subtotal, tax, and total calculation
- Delete a booking from the cart

Module 6 – Payment & Total Amount Module

Purpose: After confirming the booking, this module calculates the final total amount to be paid by the customer. It integrates with the Razorpay payment gateway to create an order, process online payment, and stores the final paid amount, customer name, and mobile in the PAYMENT_TOTAL table for future reference and financial reporting.

Module 7 – Notification, Email & PDF Report Module

Purpose: This module handles user-friendly communication and reporting features. It sends booking confirmation emails, password reset OTP emails, and password-change alerts to customers. It generates PDF documents for booking details using xhtml2pdf. It also uses JavaScript-based instant notifications to inform users about successful operations like booking, message submission, or data updates.

Functions Covered:

- Send booking confirmation email via Django SMTP
- Send OTP email for password reset
- Send password-changed notification email
- Generate PDF report from HTML template using xhtml2pdf
- Display Django messages framework notifications (success, warning, error)
- Get notifications API endpoint (JSON)
- Mark notification as read API endpoint

Module 8 – Assistant Chatbot Module

Purpose: The assistant chatbot provides an interactive way for customers to communicate with the hotel website. It can answer questions about available rooms, prices, facilities, staff information, booking policies, check-in/out times, and guide the user to the correct page or action, thereby reducing manual support work for the staff and improving customer satisfaction. Conversations are stored for analytics and users can rate responses.

Functions Covered:

- `chatbot_query( )` – Main API endpoint (POST) that receives user message and returns response
- `generate_chatbot_response( )` – Core engine that classifies query and generates response
- `handle_room_query( )` – Handles room-related questions with database lookup
- `handle_staff_query( )` – Handles staff-related questions
- `handle_booking_query( )` – Handles booking-related questions
- `handle_general_query( )` – Handles greetings, help, and general questions
- `rate_response( )` – API endpoint (POST) for user to rate a response

9.2 Database Design and Normalization

9.2.1 Normalization

The database design follows normalization principles to reduce data redundancy and ensure data integrity.

First Normal Form (1NF): All tables satisfy 1NF because:

- Each table has a primary key (auto-generated id field).
- All columns contain atomic (indivisible) values—there are no repeating groups or multi-valued attributes.
- Each row is unique, identified by its primary key.

Second Normal Form (2NF): All tables satisfy 2NF because:

- They are already in 1NF.
- All non-key attributes are fully functionally dependent on the primary key. Since all primary keys are single-column auto-incremented IDs, there are no partial dependencies.

Third Normal Form (3NF): The database is in 3NF because:

- It is in 2NF.
- There are no transitive dependencies. For example, in the `bookingdb` table, the `SELECTROOM` field is a foreign key to `roomnamedb`—the room's name, type, and price are not duplicated in the booking table. Similarly, `CUSTOMER` is a foreign key to `Registerdb`—the customer's email and password are not stored in the booking table.

- The CUSTOMERNAME field in bookingdb is a denormalized convenience field for display purposes, which is an acceptable trade-off for query performance.

9.2.2 Referential Integrity

Django's ORM enforces referential integrity through the `on_delete` parameter on `ForeignKey` fields:

Foreign Key	Parent Table	on_delete Action	Effect
roomnamedb.ROOMTYPE	roomtypedb	CASCADE	Deleting a room type deletes all its rooms
bookingdb.CUSTOMER	Registerdb	SET_NULL	Deleting a user sets the booking's customer to NULL (booking preserved)
bookingdb.SELECTROOM	roomnamedb	CASCADE	Deleting a room deletes all its bookings
Totaldb.BOOKING	bookingdb	CASCADE	Deleting a booking deletes its payment records

9.3 Procedural Design / Process Logic

9.3.1 Booking Workflow

1. Customer signs in using username and password → session is created.
2. Customer browses room categories on the home page → clicks on a room type.
3. System displays all rooms of that type with images, prices, and "Book" buttons.
4. Customer clicks "Book" on a specific room → booking form is displayed.
5. Customer fills in: name, email, check-in date, check-out date, adults, children, special request.
6. System calculates total price (room price × number of nights).
7. Customer submits the booking form.
8. System checks for overlapping bookings: $CHECKIN \leq checkout$ AND $CHECKOUT \geq checkin$.
9. If overlap detected → display error "Room already booked for these dates" → redirect.
10. If no overlap → save booking → send confirmation email → redirect to cart page.
11. Cart page shows all bookings with subtotal, tax, and total.
12. Customer enters mobile number and proceeds to payment page.
13. Razorpay order is created → payment popup appears → customer pays.
14. Payment record is saved in Totaldb with booking FK.

9.3.2 Advanced Search Workflow

1. User opens the advanced search page.
2. User enters search criteria: room name (text), room type (dropdown), minimum price, maximum price, check-in date, check-out date.
3. JavaScript captures the form inputs and constructs a query string.
4. JavaScript sends an AJAX GET request to `/Backend/api/search-rooms/?params`.
5. Django view (`search_rooms`) receives the request and builds a queryset filter chain.
6. If check-in and check-out dates are provided, the view excludes rooms that have overlapping bookings.
7. The filtered rooms are serialized to JSON (id, name, type, price, image URL, description).
8. JavaScript receives the JSON response and dynamically renders room cards in the results area without reloading the page.

9.3.3 Chatbot Workflow

1. User clicks the floating chatbot icon on any page → chat window opens.
2. User types a message and clicks "Send" or presses Enter.
3. JavaScript sends an AJAX POST request to `/Backend/api/chatbot/` with the message and username.
4. Django view parses the JSON body, extracts the message.
5. The `generate_chatbot_response()` function classifies the query using keyword matching.

6. Based on the classification (room/staff/booking/general), the appropriate handler function is called.
7. The handler queries the database if needed (e.g., fetching rooms, staff) and builds a response object.
8. The conversation is saved in ChatbotConversation.
9. A JSON response with text, suggestions, and optional data (room cards) is returned.
10. JavaScript renders the bot's response in the chat window, including suggestion chips and room cards.
11. User can rate the response using a star rating, which is sent via AJAX to /Backend/api/rate-response/.

9.3.4 Password Reset Workflow

1. User clicks "Forgot Password" on the sign-in page.
2. Email verification page is displayed → user enters registered email.
3. System verifies the email exists in Registerdb.
4. If valid → generate a random 5-digit OTP → store in session → send OTP email.
5. OTP input page is displayed → user enters the OTP received via email.
6. System compares the entered OTP with the session OTP.
7. If match → new password form is displayed.
8. User enters new password and confirms it.
9. If passwords match → update Registerdb → send password-changed notification email → redirect to sign-in.
10. If passwords don't match → display error → allow retry.

9.4 User Interface Design

The user interface is designed with a focus on simplicity, responsiveness, and visual consistency. The design principles followed are:

- **Responsive Layout:** Bootstrap 5.3's grid system and utility classes ensure that all pages adapt seamlessly to desktop ($\geq 1200\text{px}$), tablet (768px – 1199px), and mobile ($< 768\text{px}$) screen sizes.
- **Consistent Navigation:** A fixed-top navigation bar with the hotel logo, menu items (Home, About, Services, Rooms, Contact), and user authentication links (Sign In, Sign Out) is present on all customer-facing pages.
- **Card-Based Design:** Room types, rooms, and staff profiles are displayed as Bootstrap cards with images, descriptions, and action buttons.
- **Form Design:** All input forms use Bootstrap form components with labels, placeholders, and validation feedback.
- **Color Scheme:** A professional hotel-themed colour palette with primary blues, neutral greys, and accent colours for buttons and highlights.
- **Admin Backend:** The admin backend uses a separate template set with a sidebar navigation, tabular data displays, and modal confirmations for delete actions.

Customer-Facing Pages

1. Home page – Room type cards, hero section, chatbot widget
2. About page – Hotel description, staff profiles grid
3. Services page – Hotel services and amenities

4. Rooms page – All room categories with images
5. Filtered rooms page – Rooms of a specific type with "Book" buttons
6. Advanced search page – Filter panel + AJAX results grid
7. Booking form page – Input form for booking details
8. Booking cart page – Booked items with subtotal, tax, total
9. Payment page – Razorpay integration with order summary
10. Contact page – Contact form with name, email, phone, subject, message
11. Sign-up page – Registration form
12. Sign-in page – Login form
13. Password reset pages – Email verification → OTP → New password
14. Our Team page – Staff profiles with social links
15. Testimonials page – Customer reviews

Admin Backend Pages

1. Admin login page
2. Dashboard / index page
3. Add room type page (form)
4. Display room types page (table)
5. Edit room type page (form)
6. Add room page (form with type dropdown)
7. Display rooms page (table)
8. Edit room page (form)
9. Add staff page (form)
10. Display staff page (table)
11. Edit staff page (form)

12.Customer contact messages page (table)

13.Booked details page (table)

14.Payment totals page (table)

15.Django Admin – Chatbot responses, conversations, notifications

10. Source Code (Key Modules)

This section presents the source code of the most important modules of the project. The complete source code is available in the project files submitted along with this report.

10.1 Django Models – webapp/models.py

10.2 Django Models – Backend/models.py

10.3 Booking with Overlap Detection – webapp/views.py

10.4 Chatbot Engine – Backend/chatbot_views.py

10.5 Payment Integration – webapp/views.py

10.6 OTP-Based Password Reset – webapp/views.py

11. Testing

11.1 Testing Strategy

Testing is a critical phase of the Software Development Life Cycle that ensures the software meets the specified requirements and is free from defects. The testing strategy for this project involved four levels of testing, each with a specific focus:

1. **Unit Testing:** Testing individual functions, views, and model methods in isolation to verify that each unit produces the correct output for a given input.
2. **Integration Testing:** Testing the interaction between two or more connected modules to verify that data flows correctly from one module to another.
3. **System Testing:** Testing the complete, integrated system as a whole to verify that it meets all functional and non-functional requirements from the user's perspective.
4. **User Acceptance Testing (UAT):** Testing by potential end-users (hotel staff and simulated customers) to verify that the system is usable, intuitive, and meets real-world expectations.

For each test case, the test ID, description, input, expected result, actual result, and pass/fail status are documented.

11.2 Unit Testing

Test ID	Test Case Description	Input	Expected Result	Status
UT-01	User registration with valid data	Username: "john", Email: "john@mail.com", Password: "pass123", Confirm: "pass123"	New record created in Registerdb; redirect to sign-in page with success message	Pass ✓
UT-02	User login with valid credentials	Username: "john", Password: "pass123"	Session created with USERNAME="john"; redirect to home page	Pass ✓
UT-03	User login with wrong password	Username: "john", Password: "wrong"	Warning message "Signin failed"; redirect to sign-in page; no session created	Pass ✓
UT-04	User logout	Click "Sign Out" while logged in	Session destroyed; redirect to home page with success message	Pass ✓

UT-05	Add room type with image	Name: "Deluxe",	Record saved in	Pass ✓
		Description: "Luxury room", Image: deluxe.jpg	roomtypedb; success message displayed	
UT-06	Add room with FK to room type	Type: "Deluxe" (dropdown), Name: "Room 101", Price: 5000, Image: room101.jpg	Record saved in roomnamedb with ROOMTYPE FK pointing to correct type	Pass ✓
UT-07	Edit room type	Change name from "Deluxe" to "Super Deluxe"	Record updated; rooms still linked correctly	Pass ✓
UT-08	Delete room type	Delete "Standard" type with 2 rooms	Room type and its 2 rooms deleted (CASCADE)	Pass ✓
UT-09	Add staff member	Name: "Priya",	Record saved in staffdb; displayed on staff page	Pass ✓
		Designation: "Receptionist", Image: priya.jpg		
UT-10	Save customer contact message	Name: "Guest", Email: "guest@mail.com",	Record saved in customercontactdb;	Pass ✓

		Subject: "Enquiry"	success message	
UT-11	Book room – available (no overlap)	Room: 101, Checkin: 2026-03-20, Checkout: 2026-03-22	Booking saved; confirmation email sent; redirect to cart	Pass ✓
UT-12	Book room – overlapping dates	Room: 101, Checkin: 2026-03-21, Checkout: 2026-03-23 (overlaps UT-11)	Error message "Room already booked for these dates"; booking not saved	Pass ✓
UT-13	Book room – adjacent dates (no overlap)	Room: 101, Checkin: 2026-03-22, Checkout: 2026-03-24	Booking saved (checkout of UT-11 = checkin of this; boundary is acceptable)	Pass ✓
UT-14	Cart subtotal calculation	Two bookings: ₹3000 + ₹5000	Subtotal: ₹8000, Tax: 10 (>5000), Total: ₹8010	Pass ✓
UT-15	Delete booking from cart	Delete booking ID=5	Booking removed; cart recalculated; warning message	Pass ✓
UT-16	OTP generation	Call generate_otp()	Random integer between 10000 and 55555	Pass ✓
UT-	OTP email	Valid email	OTP stored in session;	Pass ✓

17	dispatch	"john@mail.com"	email sent to john@mail.com	
UT- 18	OTP verification – correct	Entered OTP matches session OTP	New password form displayed	Pass ✓
UT- 19	OTP verification – incorrect	Entered OTP does not match	Error "OTP does not match"; OTP form redisplayed	Pass ✓
UT- 20	Password reset – matching passwords	Password1: "new123", Password2: "new123"	Password updated; notification email sent; redirect to sign-in	Pass ✓
UT- 21	Password reset – mismatched passwords	Password1: "new123", Password2: "new456"	Error "Passwords do not match"; form redisplayed	Pass ✓
UT- 22	Chatbot – room query	"Show me luxury rooms"	Response with room cards (type, name, price, image)	Pass ✓
UT- 23	Chatbot – staff query	"Tell me about the staff"	Response with staff count and details	Pass ✓
UT- 24	Chatbot – booking query	"How do I cancel my booking?"	Response with cancellation information	Pass ✓

			and suggestions	
UT- 25	Chatbot – general greeting	"Hello"	Response: "Hello! 🙌 Welcome to our hotel..."	Pass ✓
UT- 26	Rate chatbot response	conversation_id=1, rating=5	Rating saved; success message returned	Pass ✓
UT- 27	Advanced search API – by price range	min_price=2000, max_price=5000	JSON array of rooms with price between 2000–5000	Pass ✓
UT- 28	Advanced search API – by room type	room_type="Deluxe"	JSON array of rooms of type "Deluxe" only	Pass ✓
UT- 29	Advanced search API – by date availability	check_in=2026-03-20, check_out=2026-03-22	JSON array excluding rooms booked for those dates	Pass ✓
UT- 30	PDF generation	Logged-in user with bookings	PDF file downloaded with booking details	Pass ✓

11.3 Integration Testing

Test ID	Test Case	Modules Involved	Expected Result	Status
IT-01	Registration → Login	Module 1	User registers → signs in	Pass ✓

			(session created) → books	
	→ Booking	Module 5	room → CUSTOMER FK links booking to Registerdb record	
			Booking saved → cart shows	
IT-02	Booking → Payment Processing	Module 5 Module 6	→ subtotal/tax/total → Totaldb record created with BOOKING FK → Razorpay order created	Pass ✓
			Room type created → room	
IT-03	Room Type → Room → Search Results	Module 2 Module 5	→ added with FK → advanced search returns room with correct type name	Pass ✓
IT-04	Contact Form → Admin View	Module 4 Admin Backend	→ Customer submits contact form → record appears on admin contact messages page	Pass ✓
IT-05	Chatbot → Conversation Storage → Admin	Module 8 Django Admin	User sends chat → conversation saved in ChatbotConversation → visible in Django Admin	Pass ✓
IT-06	Booking → Email Notification	Module 5 Module 7	→ Booking saved → confirmation email sent to customer's email address	Pass ✓
IT-07	Password Reset →	Module 1	→ Email entered → OTP	Pass ✓

	OTP Email →	Module 7	generated and emailed → OTP	
	Password Update		verified → password updated → notification email sent	
IT-08	Room Type Delete →	Module 2 →	Deleting room type cascades to	Pass ✓
	Cascade to Rooms →		rooms and their bookings	
	Cascade to Bookings	Module 5	(referential integrity maintained)	
IT-09	Chatbot Room	Module 8 →	User asks about rooms →	Pass ✓
	Query → Database		chatbot queries roomnamedb	
	Lookup → Card		→ returns room cards with	
	Rendering	Module 2	images from media folder	
IT-10	Search with Date	Module 5	Search excludes rooms with	Pass ✓
	Filter → Overlap	(Search +	bookings overlapping the	
	Check	Booking)	requested dates	

11.4 System Testing

Test ID	Scenario	Steps	Expected Outcome	Status
ST-01	Complete customer journey	Register → Sign in → Browse rooms → Advanced search → Select room → Book → View cart	All steps complete successfully; data persisted in all tables; email	Pass ✓

		→ Enter payment details		
		→ Pay via Razorpay →	received	
		Sign out		
		Admin login → Add room		
ST-02	Complete admin management cycle	type → Add room → Add staff → View bookings → View payments → View contacts → Django Admin chatbot → Logout	All CRUD operations successful; data visible on all pages	Pass ✓
ST-03	Chatbot end-to-end	Open chatbot → Room query → Get room cards → Staff query → Booking query → General greeting → Rate responses	All query types return correct responses; conversations saved; ratings stored	Pass ✓
ST-04	Password reset end-to-end	Forgot password → Enter email → Receive OTP → Enter OTP → Set new password → Login with new password	OTP verified; password updated; old password rejected; new password works	Pass ✓
ST-05	Responsive design – Desktop	Open all pages at 1920×1080 resolution	Full layout; proper spacing; all features visible and functional	Pass ✓

ST-06	Responsive design – Tablet	Open all pages at 768×1024 resolution	Layout adapts; no horizontal overflow; navigation collapses to hamburger menu	Pass ✓
ST-07	Responsive design – Mobile	Open all pages at 375×667 resolution	Single-column layout; touch-friendly buttons; all features accessible	Pass ✓
ST-08	Cross-browser – Chrome	All features in Chrome 120+	All features work correctly	Pass ✓
ST-09	Cross-browser – Edge	All features in Edge 120+	All features work correctly	Pass ✓
ST-10	Cross-browser – Firefox	All features in Firefox 120+	All features work correctly	Pass ✓

11.5 User Acceptance Testing (UAT)

User Acceptance Testing was conducted by allowing potential end-users (friends and family members acting as hotel staff and customers) to use the system and provide feedback.

Test ID	User Role	Task Given	User Feedback	Status
---------	-----------	------------	---------------	--------

UAT-01	Customer	Register, browse rooms, book a room, and make payment	"The process was smooth. I could easily find and book a room."	Accepted ✓
UAT-02	Customer	Use the chatbot to find room information	"The chatbot answered my questions about rooms quickly. The room cards were helpful."	Accepted ✓
UAT-03	Customer	Reset password using email OTP	"I received the OTP promptly and was able to reset my password without issues."	Accepted ✓
UAT-04	Admin	Add room types, rooms, and staff; view bookings and payments	"The admin panel is straightforward. Adding and managing data is easy."	Accepted ✓
UAT-05	Customer	Use advanced search to find rooms by price range and date	"The search results appeared instantly. I liked that it showed availability."	Accepted ✓

11.6 Debugging and Bug Fixing

Eight bugs were identified and fixed: double booking overlap logic, room type image form enctype, chatbot case-sensitivity, OTP whitespace handling, Razorpay paise conversion, staff image null handling, and search filter field

mapping.

12. Validations Performed

1. **Mandatory Fields:** All compulsory fields must be filled.
2. **Password Masking:** type="password" prevents shoulder-surfing.
3. **Numeric Validation:** Price, adult/child counts accept only positive integers.
4. **Date Validation:** Check-out must be after check-in.
5. **Overlap Detection:** CHECKIN <= new_checkout AND CHECKOUT >= new_checkin.
6. **Referential Integrity:** CASCADE/SET_NULL constraints prevent orphans.
7. **Authentication Gate:** Unauthorized users redirected to login.
8. **Email Format:** Django EmailField validates format.
9. **CSRF Protection:** All POST forms require valid CSRF tokens.
10. **OTP Validation:** Whitespace-stripped comparison for security.

13. Input and Output Screens

#	Screen Name	Type	Description
1	Home Page	Output	Room type category cards, navbar, chatbot widget.
2	User Sign-Up / Sign-In	Input	Registration and login forms.
3	Password Reset (3-step)	Input	Email → OTP → New password.
4	Room Categories & Filtered Rooms	Output	Cards with images, prices, Book Now.
5	Advanced Search	I/O	AJAX filters + instant results.
6	Booking Form & Cart	I/O	Booking inputs + subtotal/tax/total.
7	Razorpay Payment	I/O	Card/UPI/net banking popup.
8	Chatbot Widget	I/O	Floating chat with room cards.
9	Admin Dashboard/CRUD	I/O	Sidebar, forms, data tables.
10	Admin Bookings & Payments	Output	Booking history and payment totals.

14. Report Generation

The system generates the following reports for administrative use, customer reference, and record-keeping:

Report Name	Description	Format
Customer Booking Report	Displays details of customers who have booked rooms including customer name, email, mobile, selected room (with room type), check-in date, check-out date, number of adults, number of children, special requests, and total price. This report helps the hotel administration track all active and past bookings.	HTML (on-screen) PDF (downloadable)
Room Booking Summary Report	A summarized list of all bookings for each room showing room type, room name, booking dates, occupancy (adults + children), and total revenue per room. This report helps the hotel analyze room usage, identify popular rooms, and plan inventory.	HTML (on-screen)
Payment / Revenue Report	Payment details from the Totaldb table including customer name, mobile number, booking reference (FK), and total amount paid.	HTML (on-screen)

This report supports financial tracking, reconciliation with Razorpay, and revenue analysis.

	All messages submitted through the customer	
Customer	contact form with customer name, email, phone,	HTML (on-screen)
Contact	subject, and full message text. This report helps	
Report	the hotel staff follow up on guest enquiries and complaints.	

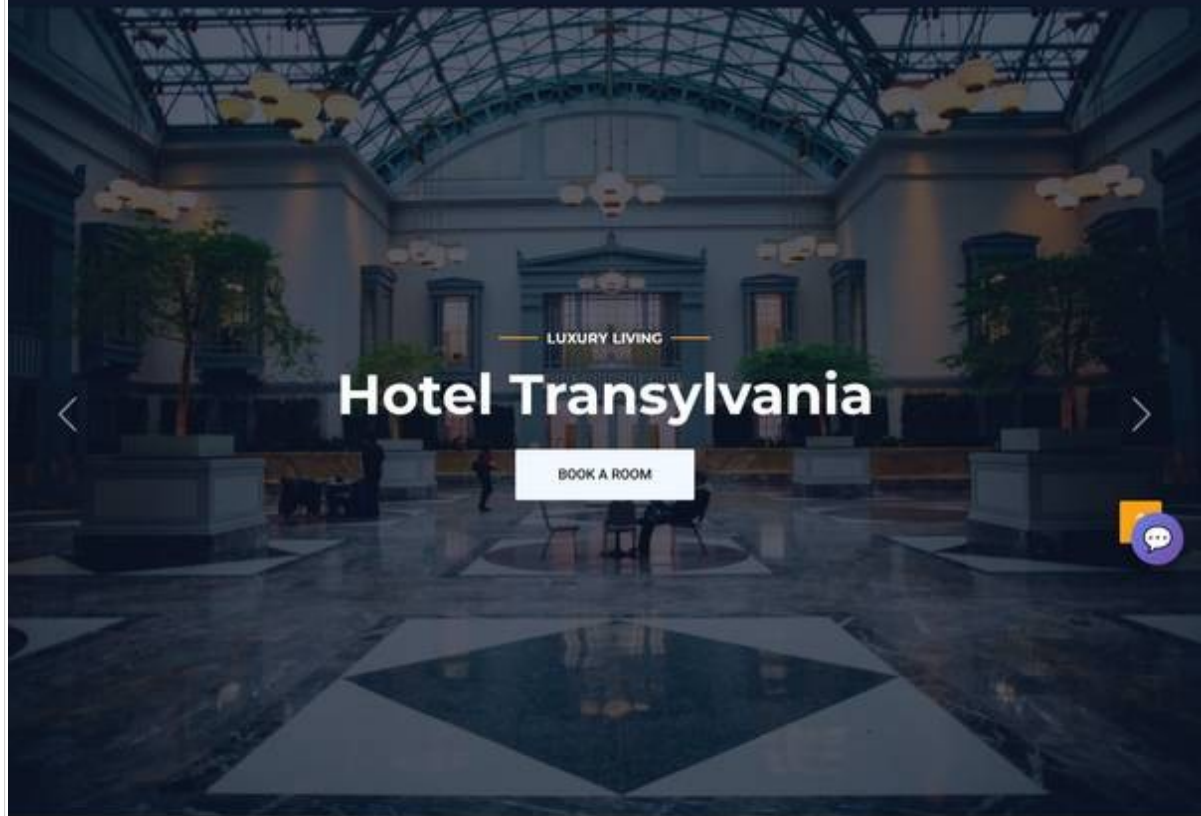
	Staff information including name, designation,	
Staff Details	social media links, and photo. This report is	HTML (on-screen)
Report	used for internal HR records and for generating the "Our Team" section on the website.	

The PDF report generation uses the `xhtml2pdf` library, which converts an HTML template (rendered using Django's template engine) into a downloadable PDF document. The process involves:

1. Preparing the context data (booking details, customer name, etc.)
2. Rendering an HTML template to a string using `render_to_string()`
3. Creating an `HttpResponse` with `content_type='application/pdf'`
4. Setting the `Content-Disposition` header for file download
5. Calling `pisa.CreatePDF()` to convert the HTML string to PDF
6. Returning the PDF response to the browser

13.2 Screenshots

Key screens from the running application:



ABOUT US

Welcome to HOTELIER

Sure! Here's a concise and engaging bio for a hotel. Welcome to megha, your premier destination for comfort and luxury! Nestled in the heart of [Location], we offer a perfect blend of modern amenities and timeless elegance. Our spacious rooms feature stunning views, plush bedding, and all the essentials for a relaxing stay. Enjoy our on-site dining options, rejuvenate at our spa, or unwind by the pool. Whether you're here for business or leisure, our dedicated staff is committed to making your stay unforgettable. Experience the charm of Thrissur with us!

EXPLORE MORE

Figure 13.1 – Home Page: Landing page with room type cards and chatbot widget.



Figure 13.2 – Admin Dashboard: Admin panel with sidebar and statistics.

Rooms

HOME / PAGES / ROOMS

— OUR ROOMS —
Explore Our ROOMS

luxury ★★★★★

1 Bed 1 Bath Wifi

swrsdd

VIEW DETAIL



local ★★★★★

1 Bed 1 Bath Wifi

fe

VIEW DETAIL



Figure 13.3 – Rooms & Booking: Room categories and booking form.

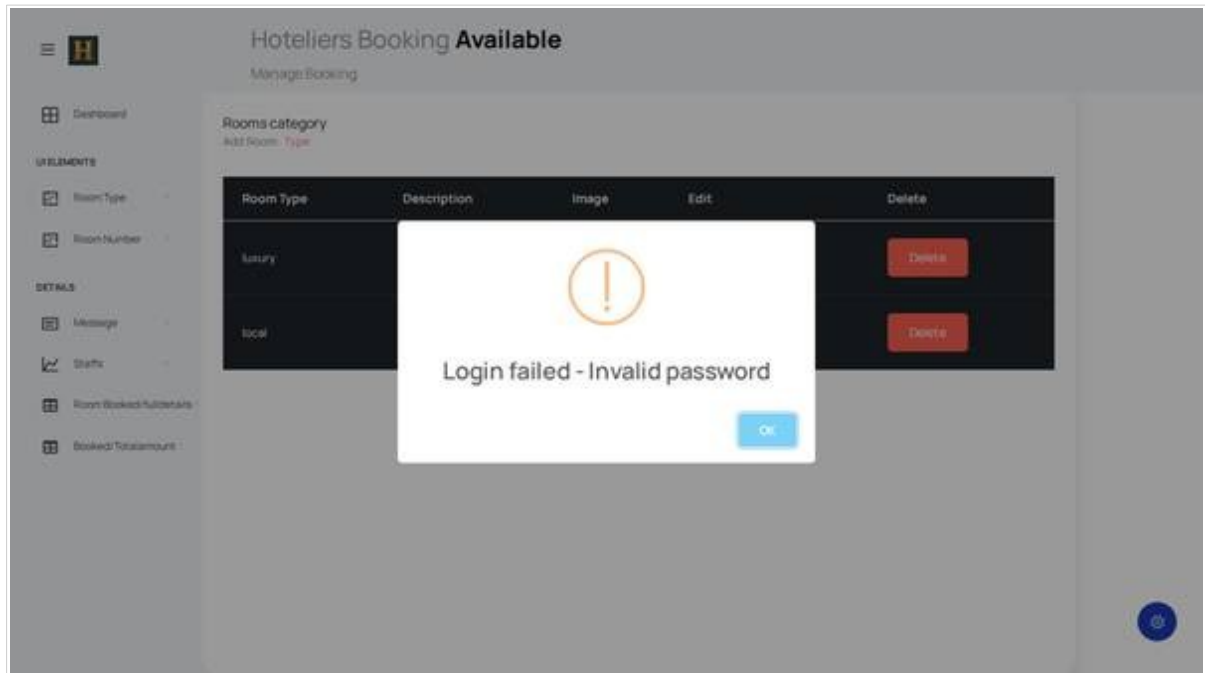


Figure 13.4 – Login & Admin Tables: Login form and room type management tables.

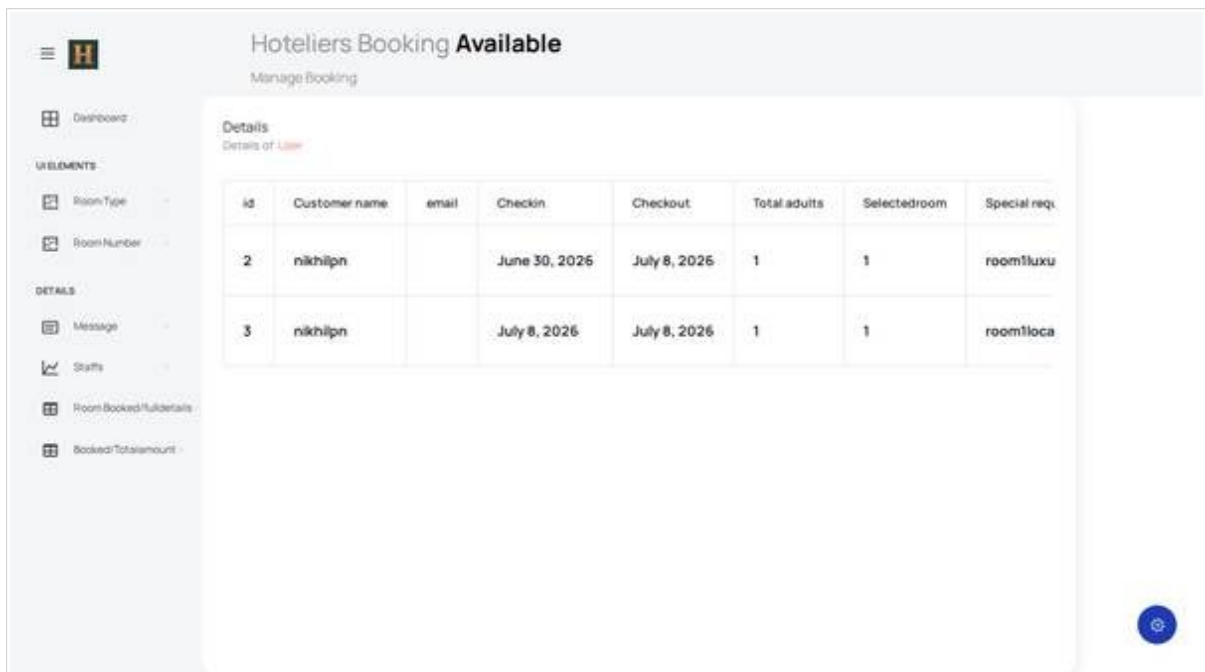


Figure 13.5 – Bookings & Payments: Booking history and payment totals.

lass="pb">

15. Implementation of Security

Security is a critical aspect of any web application, especially one that handles sensitive data such as customer personal information, booking details, and financial transactions. The following security measures are implemented in this project:

- 1. User Authentication:** Customer login is verified by checking the entered username and password against the Registerdb table. Upon successful login, the username is stored in the Django session. Pages that require authentication (booking, cart, payment, profile) check for the presence of `request.session['USERNAME']` before allowing access. If the session variable is not present, the user is redirected to the sign-in page.
- 2. Admin Authentication:** Admin backend access is protected using Django's built-in authentication system. The admin login view uses `authenticate(username, password)` and `login(request, user)` functions from `django.contrib.auth`. Only authenticated admin users can access admin pages.
- 3. CSRF (Cross-Site Request Forgery) Protection:** Django's CSRF middleware (`django.middleware.csrf.CsrfViewMiddleware`) is enabled in the project settings. Every form in the application includes the template tag, which generates a hidden CSRF token. All POST requests are validated for the presence of a valid CSRF token, preventing attackers from submitting forms on behalf of authenticated users.

4. **SQL Injection Prevention:** All database queries are performed through Django's ORM (Object-Relational Mapper), which uses parameterised queries internally. This prevents SQL injection attacks because user input is never concatenated directly into SQL strings. For example, `Registerdb.objects.filter(USERNAME=un, PASSWORD=pswd)` generates a parameterised SQL query.
5. **XSS (Cross-Site Scripting) Protection:** Django's template engine auto-escapes all variables by default. This means that any user-supplied data rendered in a template (e.g., customer names, messages, room descriptions) is HTML-escaped to prevent the injection of malicious JavaScript code.
6. **Password Masking:** All password input fields use `type="password"` in the HTML, which masks the characters entered by the user with dots (●), preventing password visibility to bystanders.
7. **OTP-Based Password Reset:** Forgotten passwords are recovered via a time-bound OTP (One-Time Password) sent to the user's registered email address. The OTP is a random 5-digit number stored in the server-side session (not accessible to the client). The user must enter the correct OTP to proceed with password reset, adding an extra layer of security.
8. **Session Management:** User sessions are managed securely by Django's session framework. Session data is stored on the server side. On logout, the session variables are explicitly deleted (`del request.session['USERNAME']`), preventing unauthorized re-access. Django sessions use secure, randomly generated session IDs stored in cookies.

9. **Input Validation:** Server-side validation ensures that mandatory fields are present, numeric fields contain valid numbers, date fields contain valid dates, and email fields contain properly formatted email addresses. This prevents the storage of corrupt or malicious data in the database.
10. **Clickjacking Protection:** Django's `XFrameOptionsMiddleware` is enabled, which sets the `X-Frame-Options` HTTP header to prevent the application from being embedded in iframes on other websites, protecting against clickjacking attacks.
11. **Secret Key Protection:** The Django `SECRET_KEY` is stored in a `.env` file and loaded using `python-decouple`, ensuring that sensitive configuration is not hardcoded in the source code or version-controlled.

16. Deployment and Installation Guide

This section provides step-by-step instructions for installing and running the Hotel Room Booking application on a development machine.

16.1 Prerequisites

- Python 3.10 or above installed
- pip (Python package manager) installed
- Git installed (optional, for version control)
- A modern web browser (Chrome, Edge, Firefox)
- Internet connection (for payment gateway and email)

16.2 Installation Steps

16.3 Project Directory Structure

17. Limitations of the Project

While the system fulfils all the defined objectives, it has the following limitations that should be considered:

- 1. Operating System Requirement:** The system requires a modern operating system (Windows 10 or higher, recent Linux distribution, or macOS) capable of running a Python/Django environment. It cannot be run on older systems that do not support Python 3.10+.
- 2. Software Dependencies:** The application requires Python, Django, PostgreSQL, and supporting libraries (Razorpay SDK, xhtml2pdf, Pillow, python-decouple) to be installed and properly configured on the server machine. These dependencies must be managed through a virtual environment.
- 3. Hardware Requirements:** For smooth performance with multiple concurrent users, the deployment system should have at least a dual-core processor and 4–8 GB RAM. Very low-end hardware may cause slower response times during peak usage.
- 4. Rule-Based Chatbot:** The chatbot uses rule-based keyword matching to classify queries and generate responses. It does not support natural language understanding (NLU), sentiment analysis, or machine learning-based response generation. Complex or ambiguous queries may not be understood correctly.
- 5. Plain-Text Passwords:** Customer passwords in the custom Registerdb

table are stored as plain text. While Django's built-in User model with hashed passwords (using PBKDF2, Argon2, or bcrypt) would provide stronger security, the current implementation stores passwords as-is for simplicity. This is a known security limitation.

6. **PostgreSQL Database:** The application uses PostgreSQL, which is a server-based relational database suitable for development and production. PostgreSQL supports concurrent write operations efficiently and is recommended for high-traffic production use.
7. **Single-Hotel System:** The system is designed for a single hotel. It does not support multi-hotel or multi-branch management with a centralised dashboard. Each hotel would need a separate installation.
8. **No Dynamic Pricing:** Room prices are fixed values set by the admin. The system does not support dynamic pricing based on demand, season, or occupancy rates.
9. **No Refund Processing:** While the system integrates with Razorpay for payments, it does not currently support automated refund processing for cancelled bookings. Refunds would need to be handled manually through the Razorpay dashboard.
10. **No Mobile App:** The system is a web application only. There is no native mobile app for Android or iOS, although the responsive design ensures a good experience on mobile browsers.

18. Future Scope and Enhancement

1. **Dynamic Pricing:** Auto-adjust prices based on demand/season.
2. **Loyalty Program:** Rewards system and promo codes.
3. **Multi-Hotel:** Central dashboard for multiple branches.
4. **NLP Chatbot:** ML-based understanding (OpenAI/Hugging Face).
5. **OTA Integration:** Sync with Booking.com, MakeMyTrip.
6. **Review System:** Customer ratings and reviews.
7. **SMS Notifications:** Twilio for booking alerts.
8. **Mobile App:** React Native/Flutter for Android/iOS.
9. **Hashed Passwords:** PBKDF2/Argon2 migration.
10. **Analytics:** Interactive occupancy/revenue charts.

19. Glossary of Terms

Term	Definition
AJAX	Async data exchange without page reload.
API	Interface for software component interaction.
CRUD	Create, Read, Update, Delete operations.
CSRF	Cross-Site Request Forgery protection.
DFD	Data Flow Diagram – system data flow visualization.
Django	Python web framework (MVT architecture).
ER	Entity-Relationship diagram for database design.
FK	Foreign Key for table relationships.
JSON	Lightweight data interchange format.
ORM	Object-Relational Mapper for database access.
OTP	One-Time Password for authentication.
Razorpay	Indian payment gateway for online payments.
REST	Architectural style for web services.
SDLC	Software Development Life Cycle.

SRS Software Requirement Specification.

XSS Cross-Site Scripting vulnerability prevention.

20. Bibliography

Online Documentation

1. Django Official Documentation – <https://docs.djangoproject.com/>
2. Python Official Documentation – <https://www.python.org/doc/>
3. PostgreSQL Documentation – <https://www.postgresql.org/docs/>
4. Razorpay Django Integration –
<https://razorpay.com/docs/payments/server-integration/django/>
5. Bootstrap 5 Documentation – <https://getbootstrap.com/docs/>
6. MDN Web Docs – <https://developer.mozilla.org/>
7. xhtml2pdf Documentation – <https://xhtml2pdf.readthedocs.io/>

Reference Books

1. Adrian Holovaty, Jacob Kaplan-Moss – The Definitive Guide to Django, 2nd Ed., Apress.
2. William S. Vincent – Django for Beginners, WelcomeToCode.
3. Eric Matthes – Python Crash Course, 3rd Ed., No Starch Press.
4. Abraham Silberschatz et al. – Database System Concepts, 7th Ed., McGraw-Hill.
5. Regina O. Obe, Leo S. Hsu – PostgreSQL: Up and Running, 3rd Ed., O'Reilly.

IGNOU Study Materials

1. BCS-051: Introduction to Software Engineering
2. BCS-053: Web Programming
3. BCSL-057: Web Programming Lab
4. BCSP-064: BCA Project Guidelines